Prueba Técnica — Developer · OKY

Contexto de la prueba

> "En OKY desarrollamos interfaces de usuario para una billetera digital que sirve a usuarios en Centroamérica. Esta prueba simula el tipo de trabajo que harías en tu primera semana: tomar un requerimiento, consumir datos de un servicio externo, construir componentes reutilizables y entregar algo que funcione en producción. No buscamos perfección — buscamos ver cómo piensas, cómo estructuras tu código y cómo tomas decisiones cuando hay tiempo limitado."

El requerimiento

Construye una mini-aplicación web llamada **OKY Wallet — Transaction Explorer**.

Simula una pantalla de historial de movimientos de una billetera digital consumiendo la API pública de SpaceX como fuente de datos GraphQL:

- Explorer: https://studio.apollographql.com/public/SpaceX-pxxbxen/variant/current/explorer
- O cualquier otra API GraphQL pública de tu elección.

> **Nota sobre los datos:** Puedes usar los datos que devuelve la API libremente — no importa si son cohetes o misiones espaciales. Lo que evaluamos es cómo los manejas y presentas, no el dominio de negocio.

Funcionalidades requeridas

Req 1 — Lista paginada

Muestra una lista de registros consumidos desde GraphQL. Soporta paginación (puede ser "cargar más" o paginación numérica). Muestra al menos 4 campos por registro.

Req 2 — Filtro o búsqueda

Al menos un filtro funcional que modifique la query GraphQL o filtre los resultados del lado del cliente. Debe quedar claro cuál es el enfoque elegido y por qué.

Req 3 — Vista de detalle

Al hacer click en un registro, muestra información adicional en un panel lateral, modal o nueva vista. La navegación debe ser manejada con React Router o el sistema de routing del framework elegido.

Req 4 — Estados completos

La UI debe manejar correctamente:

- Estado de carga (skeleton o spinner)
- Estado de error (mensaje claro con opción de reintentar)
- Estado vacío (cuando no hay resultados)

Req 5 — TypeScript estricto

Todo el proyecto en TypeScript. Sin any explícito. Los tipos de los datos de la API deben estar definidos.

Funcionalidades opcionales (suman puntos)

Opcional A — Diseño responsivo

La app se ve bien en móvil (375px) y desktop (1280px). No necesita ser perfecta, pero tampoco rota.

Opcional B — Tests

Al menos 2 tests de componentes con Testing Library o Vitest. No necesitan cubrir todo — elige los más críticos y explica por qué los elegiste.

Opcional C — Optimistic UI o cache

Implementa alguna estrategia de Apollo Client cache (fetch policy, refetch, optimistic response) y documenta qué hiciste y por qué.

Opcional D — Accesibilidad básica

Uso correcto de roles ARIA, navegación por teclado en la lista y el detalle, contraste adecuado.

Stack permitido

Obligatorio:

- React (cualquier versión)
- TypeScript
- Apollo Client para el consumo de GraphQL
- Gatsby.js

Libre elección:

- Framework de routing
- Librería de estilos (Tailwind, styled-components, CSS modules, etc.)
- Librería de componentes UI si lo desea

No permitido:

- Usar un template o boilerplate descargado de internet como base del proyecto.

Entregables

1. Repositorio GitHub

URL del repositorio. Debe tener historial de commits con mensajes descriptivos. No se aceptan force-push que borren el historial.

2. Demo en vivo

URL de la app desplegada en Vercel, Netlify, Railway o similar. Debe cargar sin pasos adicionales.

3. README.md con estas secciones obligatorias

Cómo correr el proyecto Instrucciones para correr localmente en 3 pasos o menos.

Decisiones técnicas ¿Por qué elegiste ese stack? ¿Qué patrón de estado usaste y por qué?

Trade-offs ¿Qué sacrificaste por tiempo? ¿Qué harías diferente con una semana más?

Lo que más te costó De la prueba y cómo lo resolviste.